Representation Learning for rankings

Lecture 4: From Bag-of-Words to Contextualized Representations

Avishek Anand

Information Retrieval

February 13, 2026

Outline

1. From Sparse to Dense — Why We Need Embeddings
2. The Transformer Architecture
3. BERT for Information Retrieval
4. Training Cross-Encoders in Practice
5. The Two-Stage Retrieve-then-Rerank Pipeline
6. Hands-On with PyTerrier
7. Connections and Summary
8. Summary and Assignment

Section 1

From Sparse to Dense — Why We Need Embeddings

Recap: Where We Are

What We've Built So Far

- **Lecture 2:** Inverted indexes and ANN indexes for efficient retrieval
- **Lecture 3:** BM25, Language Models, RM3 for scoring and expansion

The Persistent Problem

All our methods are **bag-of-words**:

- "car problems" and "automobile issues" — BM25 score = 0
- "bank" (financial) vs. "bank" (river) — same representation
- Word order ignored: "dog bites man" = "man bites dog"

Today's Goal

Represent words, queries, and documents as **dense vectors** that capture meaning.

The Representation Gap

Sparse Representation
$[0, 0, \ldots, 3, \ldots, 0, 2, \ldots, 0]$
$\in \mathbb{R}^{|V|} \quad (|V| \approx 30{,}000)$
$>$99.9% zeros

This lecture →

Dense Representation
$[0.21, -0.05, 0.83, \ldots]$
$\in \mathbb{R}^{d} \quad (d = 768)$
100% non-zero

Each dimension = one word
Lexical matching only

Each dimension = latent feature
Semantic matching

Key Question

How do we learn dense representations that capture word meaning?

The Distributional Hypothesis

Core Principle (Firth, 1957)

"You shall know a word by the company it keeps."

Example: What do we know about the word "*tezguino*"?

- "A bottle of *tezguino* is on the table."
- "Everybody enjoys *tezguino*."
- "We brew *tezguino* from corn."

Without ever seeing a definition, we infer: *tezguino* is an alcoholic beverage.

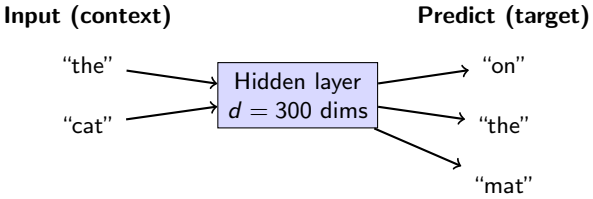
Implication for IR

Words that appear in similar contexts should have similar vector representations. If we achieve this, "car" and "automobile" will be close in vector space.

Word2Vec: Words as Vectors

Key Idea (Mikolov et al., 2013)

Train a neural network to predict context words from a target word (Skip-gram) or a target word from context (CBOW). The learned weight matrix *is* the word embedding.



After training: each word gets a d-dimensional vector. Words in similar contexts $\rightarrow$ similar vectors.

Word2Vec: Famous Properties

Linear Analogies

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

What this tells us:

- Word vectors capture **relational structure**
- Gender, tense, plurality encoded as directions in vector space
- Semantic similarity $\approx$ vector proximity (cosine similarity)

Word2Vec: Relevance to IR

- $\cos(\vec{\text{car}}, \vec{\text{automobile}}) \approx 0.7$ — high similarity!
- $\cos(\vec{\text{car}}, \vec{\text{banana}}) \approx 0.1$ — low similarity
- Could we use these for retrieval? *Yes, but...*

Limitation

Each word has **exactly one** vector, regardless of context.
"bank" (river) = "bank" (financial) — same vector!

From Words to Documents: The Naive Approach

Average Word Embeddings

$$\vec{d} = \frac{1}{|d|} \sum_{w \in d} \vec{w}$$

Example: "the cat sat on the mat"

$$\vec{d} = \frac{\vec{\text{the}} + \vec{\text{cat}} + \vec{\text{sat}} + \vec{\text{on}} + \vec{\text{the}} + \vec{\text{mat}}}{6}$$

✓ **Advantages:**

- Simple and fast
- Captures some semantics
- Fixed-size output for any doc

✗ **Problems:**

- Loses word order completely ("dog bites man" = "man bites dog")
- Polysemy unresolved
- Rare/important words drowned out

Static vs. Contextualized Embeddings

Static Embeddings
Word2Vec, GloVe
"bank" $\rightarrow$ one vector always
Context-independent

$\xrightarrow{\textbf{2018}}$

Contextualized Embeddings
ELMo, BERT, GPT
"bank" $\rightarrow$ different vector depending on sentence
Context-dependent

Example:

- "I deposited money at the **bank**." $\rightarrow \vec{v}^{(1)}_{\text{bank}}$ (financial)
- "We had a picnic by the river **bank**." $\rightarrow \vec{v}^{(2)}_{\text{bank}}$ (riverside)
- $\vec{v}^{(1)}_{\text{bank}} \neq \vec{v}^{(2)}_{\text{bank}}$ — polysemy resolved!

Key Insight

Contextualized embeddings solve both synonymy *and* polysemy.

Section 2

The Transformer Architecture

Why Transformers?

Before Transformers (pre-2017)

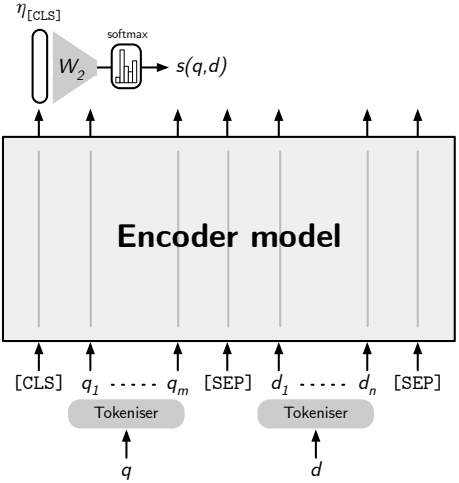
Sequential models (RNNs, LSTMs):

- Process words one at a time, left to right
- Long-range dependencies are hard (vanishing gradients)
- Cannot parallelize training (each step depends on previous)

The Transformer Insight (Vaswani et al., 2017) in "Attention Is All You Need"

- Process all words **simultaneously** (parallel)
- Each word attends to **every other word** directly
- No sequential bottleneck $\rightarrow$ massively faster training
- Result: the foundation of BERT, GPT, and all modern NLP

Encoder Model Architecture



Self-Attention: The Core Mechanism

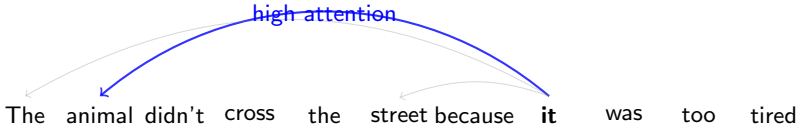
Intuition

For each word in the input, self-attention asks: "Which other words should I pay attention to in order to understand the meaning of *this* word?"

Example: "The animal didn't cross the street because **it** was too tired."

What does "it" refer to? **The animal.**

Self-attention learns to connect "it" $\rightarrow$ "animal" by assigning a high attention weight between them.



Self-Attention: Query, Key, Value

Formal Mechanism

Each word w_i is projected into three vectors:

- **Query** $\vec{q}_i$: "What am I looking for?"
- **Key** $\vec{k}_i$: "What do I contain?"
- **Value** $\vec{v}_i$: "What information do I provide?"

Attention computation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Step by step:

1. Compute similarity: $\vec{q}_i \cdot \vec{k}_j$ for all pairs (i, j)
2. Scale by $\sqrt{d_k}$ to prevent extreme values,
3. apply softmax $\rightarrow$ attention weights (sum to 1)

Self-Attention: Worked Example (1/2)

Sentence: "`retrieval models`" (2 tokens for simplicity, $d_k = 2$)

Projections (learned weight matrices):

$$\vec{q}_{\text{retrieval}} = [1.0, 0.5] \qquad \vec{q}_{\text{models}} = [0.3, 1.2]$$
$$\vec{k}_{\text{retrieval}} = [0.8, 0.2] \qquad \vec{k}_{\text{models}} = [0.1, 1.0]$$
$$\vec{v}_{\text{retrieval}} = [0.3, 0.9] \qquad \vec{v}_{\text{models}} = [0.7, 0.4]$$

Step 1: Dot products ($\vec{q}_i \cdot \vec{k}_j$):

$$\begin{aligned} \text{retrieval} \rightarrow \text{retrieval}: &\quad 1.0 \times 0.8 + 0.5 \times 0.2 = 0.9 \\ \text{retrieval} \rightarrow \text{models}: &\quad 1.0 \times 0.1 + 0.5 \times 1.0 = 0.6 \end{aligned}$$

Step 2: Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.41$: $\quad [0.64, 0.43]$

Self-Attention: Worked Example (2/2)

Recall: Computing attention for "retrieval" in "retrieval models"

After scaling: $[0.64, 0.43]$

Step 3: Softmax (normalize to probabilities):

$$\text{softmax}([0.64, 0.43]) = [\,0.55,\ 0.45\,]$$

These are the **attention weights**: how much "retrieval" attends to each token.

Step 4: Weighted sum of values:

$$\begin{aligned}\vec{h}_{\text{retrieval}} &= 0.55 \times \vec{v}_{\text{retrieval}} + 0.45 \times \vec{v}_{\text{models}} \\ &= 0.55 \times [0.3, 0.9] + 0.45 \times [0.7, 0.4] \\ &= [0.48, 0.68]\end{aligned}$$

Result

Multi-Head Attention

Problem

A single attention head can only capture one type of relationship. But words relate to each other in multiple ways (syntactic, semantic, positional).

Solution: Multiple Heads

Run h separate attention computations in parallel, each with its own Q, K, V projections. Concatenate and project:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot W^O$$

BERT uses $h = 12$ heads. Each head can learn different patterns:

- Head 1: subject-verb relationships
- Head 2: adjective-noun relationships
- Head 3: coreference ("it" $\rightarrow$ "animal")
- Head 4: positional/sequential patterns

Positional Encoding

Problem

Self-attention is permutation-invariant — "dog bites man" and "man bites dog" produce the same attention weights! Word order is lost.

Solution: Add Position Information

Before feeding tokens to the transformer, add a **positional encoding** vector to each word embedding:

$$\vec{x}_i = \vec{e}_{\text{word}_i} + \vec{p}_i$$

Sinusoidal encoding (Vaswani et al.):

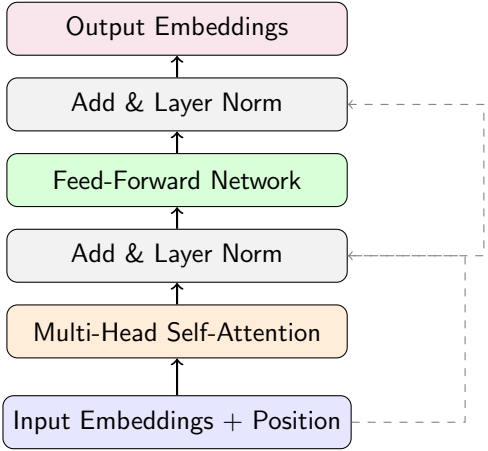
$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

Each position gets a unique pattern. Nearby positions have similar encodings.

BERT uses learned positional embeddings instead (up to 512 positions).

The Transformer Encoder Block

BERT:
12 layers stacked
768 dims
12 attention heads
110M parameters



Residual connections allow gradients to flow and stabilize training

Section 3

BERT for Information Retrieval

BERT: Pre-training

Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

A transformer encoder pre-trained on massive text with two objectives:

1. Masked Language Modeling (MLM)

- Randomly mask 15% of tokens
- Predict masked tokens from context
- Bidirectional (sees left *and* right)

Input: "The [MASK] sat on the mat"
Predict: "cat" (from context)

2. Next Sentence Prediction (NSP)

- Given sentence A, is sentence B the actual next sentence?
- Binary classification
- Learns inter-sentence relationships

A: "The cat sat on the mat."
B: "It was very comfortable." ✓

Pre-training Data

BooksCorpus (800M words) + English Wikipedia (2.5B words). Trained for 4 days on

BERT: Why It Matters for IR (1/2)

Key Properties

1. **Bidirectional context:** Every token sees all other tokens (not just left-to-right like GPT)
2. **Deep:** 12 layers of attention $\rightarrow$ rich, hierarchical representations
3. **Pre-trained:** Learned from billions of words $\rightarrow$ knows language
4. **Fine-tunable:** Adapt to specific tasks with small labeled data

Transfer Learning

BERT brings the power of transfer learning to NLP: pre-train once on massive data, fine-tune for specific tasks with minimal labeled data.

BERT: Why It Matters for IR (2/2)

For IR, BERT can:

- Understand that "automobile issues" is related to "car problems"
- Disambiguate "bank" based on surrounding context
- Capture negation: "not relevant" $\neq$ "relevant"
- Handle word order: "python eats mouse" $\neq$ "mouse eats python"

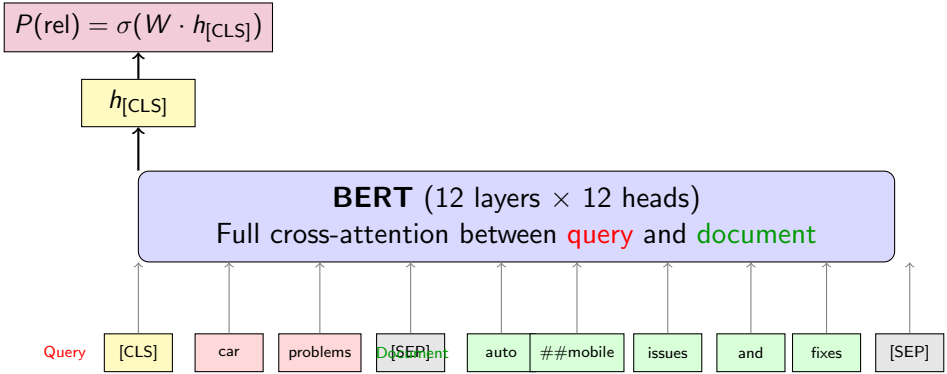
Recall

None of this was possible with BM25 or static embeddings.

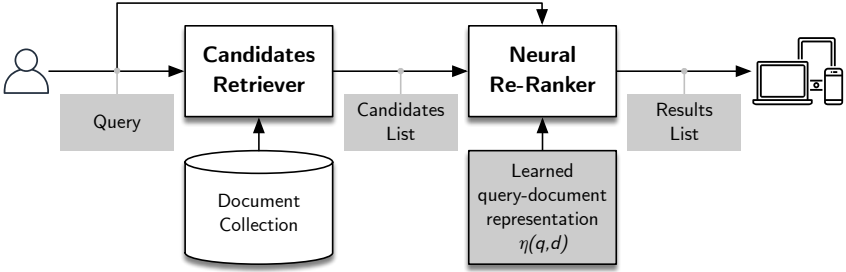
Cross-Encoder: BERT for Re-ranking

Architecture (Nogueira & Cho, 2019)

Feed the query and document **together** through BERT. Use the [CLS] token output as a relevance score.



Cross-Encoder Architecture (Detailed)



Cross-Encoder: Why It Works

Full Cross-Attention

Every query token attends to every document token (and vice versa) through all 12 layers. This enables:

1. **Soft matching:** "car" attends to "automobile" $\rightarrow$ learns they're related
2. **Context understanding:** "not working" is different from "working"
3. **Relevance patterns:** The model learns what makes a document relevant to a query, not just topical overlap

✓ **Strengths:**

- Most accurate neural ranking
- Rich query-document interaction
- Pre-trained knowledge

✗ **Limitations:**

- Must encode *every* (q, d) pair
- Cannot pre-compute doc embeddings
- Too slow for full-corpus retrieval

Cross-Encoder: Latency Analysis

How slow is "too slow"?

Candidates	Time/pair	Total	Feasible?
1,000,000 (full corpus)	5 ms	5,000 s	✗ No
10,000	5 ms	50 s	✗ No
1,000	5 ms	5 s	Borderline
100	**5 ms**	**0.5 s**	✓Yes
20	5 ms	0.1 s	✓Yes

Conclusion

Cross-encoders can realistically re-rank the **top 100–1000** documents from a first-stage retriever. They **cannot** replace the retriever itself.

Worked Example: Cross-Encoder Scoring (1/2)

Query: "side effects of aspirin"

Doc A: "Aspirin may cause stomach bleeding and ringing in the ears as common adverse reactions."

Doc B: "The company's stock price went up after the aspirin sales report."

Step 1: Tokenization (WordPiece):

- `[CLS] side effects of aspirin [SEP] aspirin may cause stomach bleeding and ringing in the ears as common adverse reactions . [SEP]`
- `[CLS] side effects of aspirin [SEP] the company ' s stock price went up after the aspirin sales report . [SEP]`

Step 2: BERT processes each pair (full cross-attention between query and document tokens)

Worked Example: Cross-Encoder Scoring (2/2)

Query: "side effects of aspirin"

Step 3: Classification head on [CLS]:

- **Doc A:** "adverse reactions", "bleeding", "stomach" $\rightarrow$ high attention to "side effects"
  - $P(\text{rel}) = \mathbf{0.94}$ ✓
- **Doc B:** "stock price", "sales report" $\rightarrow$ low attention to "side effects"
  - $P(\text{rel}) = \mathbf{0.12}$ ✗

Why This Works

BM25 might rank Doc B highly too (it contains "aspirin"). The cross-encoder understands that the *topic* doesn't match the query intent.

Fine-Tuning BERT for Ranking (1/2)

Training Data

Pairs of (query, document) with relevance labels:

- **MS MARCO:** 530K queries with sparse judgments (1 relevant passage per query)
- **TREC:** Dense judgments (multiple levels: 0, 1, 2, 3)

Training objective:

- **Pointwise:** Binary cross-entropy on $P(\text{rel})$ for each (q, d)
  - Simpler, treats each pair independently
- **Pairwise:** Given (q, d^+, d^-), train so $\text{score}(q, d^+) > \text{score}(q, d^-)$
  - Better ranking performance

Fine-Tuning BERT for Ranking (2/2)

Practical fine-tuning setup:

- Start from pre-trained BERT checkpoint (e.g., `bert-base-uncased`)
- Fine-tune on IR data for 2–3 epochs
- Learning rate: 2×10^{-5} (small, to preserve pre-trained knowledge)
- Max sequence length: 512 tokens
  - Query $\approx$ 10 tokens + Document $\approx$ 500 tokens

Training Time

Fine-tuning BERT-base on MS MARCO: $\sim$12–24 hours on a single GPU.

Key Point

We're **fine-tuning**, not training from scratch. This leverages BERT's pre-trained language understanding.

monoBERT and monoT5

monoBERT

- BERT-base or BERT-large
- Classification head on [CLS]
- Input: [CLS] q [SEP] d [SEP]
- Output: $P(\text{relevant})$
- **110M–340M parameters**

Nogueira & Cho (2019)

monoT5

- T5 (encoder-decoder)
- Input: "Query: q Document: d Relevant:"
- Output: "true" or "false"
- Score $= P(\text{"true"})$
- **220M–3B parameters**

Nogueira et al. (2020)

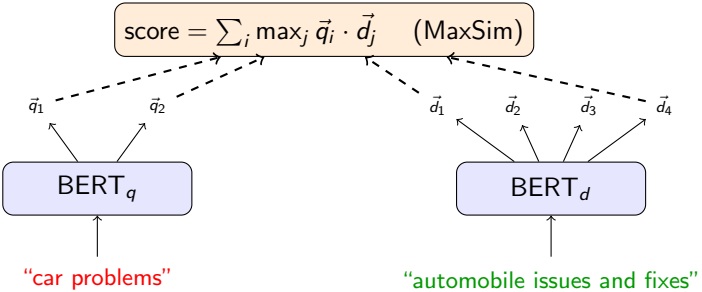
Comparison

- monoT5 (3B) slightly outperforms monoBERT on most benchmarks
- Both are cross-encoders (same speed limitation)

ColBERT: Late Interaction (Preview)

Key Idea (Khattab & Zaharia, 2020)

A middle ground between cross-encoders and bi-encoders: encode query and document **separately**, but compare at the **token level** using MaxSim.



ColBERT: Why It Matters

MaxSim Scoring

For each query token q_i, find the document token d_j with highest cosine similarity. Sum across all query tokens:

$$\text{score}(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \vec{q}_i \cdot \vec{d}_j$$

	Cross-Encoder	**ColBERT**	**Bi-Encoder**
Interaction	Full (all layers)	Late (token-level)	None (single vector)
Pre-compute docs	✗ No	✓ Yes (per-token)	✓ Yes (single vec)
Use for retrieval	✗ No	✓ With ANN	✓ With ANN
Accuracy	Highest	Near cross-encoder	Lower
Storage per doc	N/A	$\lvert d\rvert \times 128$ floats	1×768 floats

Section 4

Training Cross-Encoders in Practice

MS MARCO: The Standard Training Dataset

MS MARCO Passage Ranking (Nguyen et al., 2016)

- **8.8M passages** from web documents
- **530K training queries** with sparse relevance labels
- Each query has $\sim$1 relevant passage (sometimes 2–3)
- **6,980 dev queries** for validation

Scale

This is the largest publicly available passage ranking dataset, widely used for training neural rankers.

MS MARCO: The Sparse Labels Challenge

Key Challenge: Sparse Labels

For each query, only $\sim$1 passage is labeled relevant out of 8.8M. That means:

- Many relevant passages are **unlabeled** (false negatives)
- We can't treat all unlabeled passages as truly non-relevant
- Negative sampling strategy matters enormously

Implication for training:

- Random negatives are too easy
- Need **hard negatives** from BM25 top-k
- This forces the model to learn beyond lexical matching

Training Pipeline: Step by Step

1. **Generate training triples:** $(q,\ d^+,\ d^-)$
2. **Tokenize:** [CLS] q [SEP] d [SEP], pad/truncate to 512
3. **Batch:** group into mini-batches of 16–32 pairs
4. **Forward pass:** BERT encodes each (q, d) pair
5. **Compute loss:** binary cross-entropy or pairwise
6. **Backprop + update:** AdamW, linear warmup + decay

Typical training: 2–3 epochs over MS MARCO triples, ∼12–24 hours on a single GPU.

Negative Sampling Strategies

The Core Question

Query has 1 labeled positive. Where do the negatives come from?

1. **Random negatives**
   - Sample random passage from corpus
   - Easy for model to distinguish → weak training signal
   - ✗ Not recommended for production
2. **BM25 hard negatives** ✓ *Recommended*
   - Run BM25, take top-k *non-relevant* passages
   - These are topically related but not relevant → strong signal
   - Forces model to learn beyond term overlap

Training Hyperparameters

Batching & Memory

- Max seq length 512 tokens
- Batch size 16–32 (limited by GPU)
- **Gradient accumulation:** simulate larger batches (e.g., 4 steps $\times$ batch 8 $=$ effective 32)
- Mixed precision (fp16) cuts memory $\sim$50%
- A single 24GB GPU can train BERT-base comfortably

Optimization

- Learning rate: 1–3 $\times 10^{-5}$
- Warmup: 10% of steps
- Scheduler: linear decay
- Optimizer: AdamW ($\epsilon = 10^{-6}$)
- Epochs: 2–3 (more $\rightarrow$ overfitting)
- Evaluate every 5–10K steps

Typical Training Time

2–3 epochs over MS MARCO triples: $\sim$12–24 hours on a single GPU.

Common Training Mistakes

Avoid These Pitfalls

1. **Too many epochs**
   - Cross-encoders overfit quickly on MS MARCO
   - 3 epochs is usually the max
   - Watch dev set performance carefully
2. **Learning rate too high**
   - Catastrophic forgetting of pre-trained knowledge
   - Stay $\leq 3 \times 10^{-5}$
   - Use warmup to stabilize early training
3. **Only random negatives**
   - Model never sees hard cases
   - Always use BM25 hard negatives
4. **Ignoring document truncation**
   - Long passages get cut at 512 tokens
   - Consider first-n or sliding-window strategies

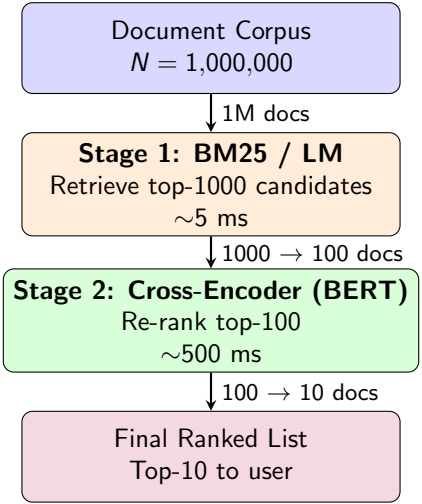
Training Code Sketch (Hugging Face)

```python
from transformers import AutoModelForSequenceClassification
from transformers import AutoTokenizer, Trainer, TrainingArguments

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2)
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

def tokenize_pair(example):
    return tokenizer(
        example["query"], example["passage"],
        truncation=True, max_length=512, padding="max_length")

# training_args controls batching, lr, schedule
args = TrainingArguments(
    output_dir="./cross-encoder",
    per_device_train_batch_size=16,
    gradient_accumulation_steps=2,  # effective batch 32
    learning_rate=2e-5,
    num_train_epochs=3,
    warmup_ratio=0.1,
    fp16=True,
    evaluation_strategy="steps",
    eval_steps=10000,
)

trainer = Trainer(model=model, args=args,
    train_dataset=train_ds, eval_dataset=dev_ds)
trainer.train()
```

Section 5

The Two-Stage Retrieve-then-Rerank Pipeline

The Two-Stage Pipeline



Why Two Stages?

Stage 1: BM25 (Recall-Oriented)

- **Speed:** microseconds per query
- **Role:** cast a wide net
- Catch as many relevant docs as possible
- Some irrelevant docs are fine
- Metric: **Recall@1000**

Stage 2: BERT (Precision-Oriented)

- **Speed:** $\sim$5 ms per doc
- **Role:** sort precisely
- Push truly relevant docs to top
- Remove false positives from BM25
- Metric: **NDCG@10**

Critical Dependency

The re-ranker can only re-order what Stage 1 retrieves. If BM25 misses a relevant document, the cross-encoder can **never** recover it.

Recall ceiling from Lecture 2: Stage 1 determines the ceiling for the entire pipeline.

Pipeline Variants

Pipeline	NDCG@10	Latency	Notes
BM25 only	0.45	5 ms	Baseline
BM25 + RM3	0.49	15 ms	Query expansion
BM25 + monoBERT	0.55	500 ms	Neural re-rank
BM25 + RM3 + monoBERT	**0.57**	520 ms	Best sparse+neural
BM25 + monoT5-3B	**0.58**	2 s	Larger model

(Approximate figures on MS MARCO passage ranking)

Observations

- Cross-encoder adds ∼10 NDCG points over BM25 — large improvement!
- RM3 still helps even with BERT (better Stage 1 recall)
- Larger models help but with diminishing returns and higher latency

What the Cross-Encoder Fixes

Three categories where BERT improves over BM25:

1. **Vocabulary Mismatch**
   - Query: "heart attack" $\rightarrow$ BM25 misses "myocardial infarction"
   - BERT: cross-attention connects "heart attack" to "myocardial infarction" ✓
2. **Semantic Relevance vs. Topical Overlap**
   - Query: "is coffee bad for you" $\rightarrow$ BM25 ranks doc about coffee brands highly
   - BERT: understands the *health* intent ✓
3. **Negation and Nuance**
   - Query: "countries without death penalty" $\rightarrow$ BM25 matches "death penalty"
   - BERT: understands "without" changes the intent ✓

But Still Fails When...

BM25 doesn't retrieve the relevant document in top-1000 at all (recall ceiling!).

Section 6

Hands-On with PyTerrier

Setup: Loading Index and Models

```python
import pyterrier as pt
if not pt.started():
    pt.init()

dataset = pt.get_dataset('irds:vaswani')
index = pt.IndexFactory.of("./vaswani_index")
topics = dataset.get_topics().head(10)
qrels = dataset.get_qrels()

# BM25 baseline (from Lecture 3)
bm25 = pt.BatchRetrieve(index, wmodel="BM25")

# For neural re-ranking, we use PyTerrier's
# integration with Hugging Face models
from pyterrier_t5 import MonoT5ReRanker

# Load monoT5 re-ranker (small version for demo)
monoT5 = MonoT5ReRanker(model='castorini/monot5-base-msmarco')
```

Exercise 1: BM25 vs. Cross-Encoder

```
# Stage 1: BM25 retrieves top 1000
# Stage 2: monoT5 re-ranks top 100
pipeline_rerank = (bm25 % 100) >> monoT5

# Compare
comparison = pt.Experiment(
    [bm25, pipeline_rerank],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "P_10"],
    names=["BM25", "BM25 + monoT5"]
)
print(comparison)

# Per-query analysis: which queries improve most?
results_bm25 = pt.Experiment([bm25], topics, qrels,
    eval_metrics=["ndcg_cut_10"], perquery=True)
results_rerank = pt.Experiment([pipeline_rerank], topics, qrels,
    eval_metrics=["ndcg_cut_10"], perquery=True)
```

Exercise 2: Pipeline Combinations

```
# Build multiple pipelines
rm3 = pt.rewrite.RM3(index)

pipelines = {
    "BM25": bm25,
    "BM25 + RM3": bm25 >> rm3 >> bm25,
    "BM25 + monoT5": (bm25 % 100) >> monoT5,
    "BM25 + RM3 + monoT5": (
        bm25 >> rm3 >> bm25) % 100 >> monoT5,
}

results = pt.Experiment(
    list(pipelines.values()), topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "recall_100"],
    names=list(pipelines.keys())
)
print(results)
```

Exercise 3: Effect of Re-rank Depth

```python
# How many docs should we re-rank?
depths = [10, 20, 50, 100, 200, 500]
pipelines_depth = {
    f"rerank@{d}": (bm25 % d) >> monoT5
    for d in depths
}

results = pt.Experiment(
    [bm25] + list(pipelines_depth.values()),
    topics, qrels,
    eval_metrics=["ndcg_cut_10", "recall_100"],
    names=["BM25 only"] + list(pipelines_depth.keys())
)
print(results)

# Plot: NDCG@10 vs re-rank depth
# Observe: diminishing returns after ~100
# Also: recall_100 stays same (only reordering!)
```

Student Exercises

Exercise A: Analyze Cross-Encoder Improvements (10 min)

- Find a query where monoT5 significantly improves over BM25
- Inspect the top-5 documents from each
- Why did BM25 fail? (vocabulary mismatch? topical but not relevant?)

Exercise B: When Does Re-ranking Fail? (10 min)

- Find a query where monoT5 does **not** help (or hurts)
- Check: is the relevant document in BM25's top-1000? (recall ceiling)
- What would fix this? (hint: dense retrieval, Lecture 5)

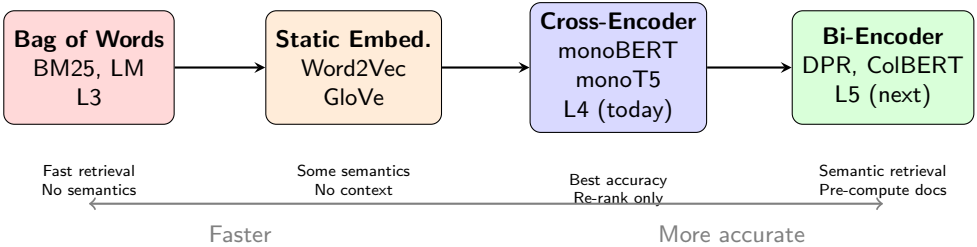
Exercise C: Latency Measurement (5 min)

- Time BM25 alone vs. BM25+monoT5 per query
- How does re-rank depth affect latency?

Section 7

Connections and Summary

The Representation Spectrum

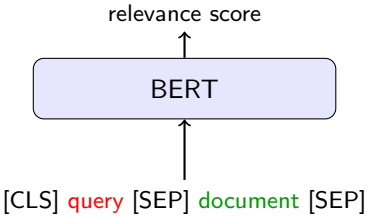


Today's Key Insight

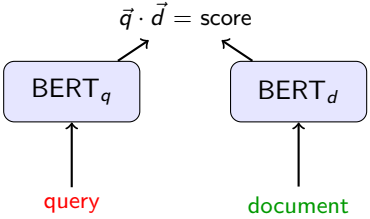
Cross-encoders are the **most accurate** ranking method but **cannot retrieve**. They must be paired with a fast first stage (BM25 or dense retrieval).

Cross-Encoder vs. Bi-Encoder (Preview of Lecture 5)

Cross-Encoder (today)

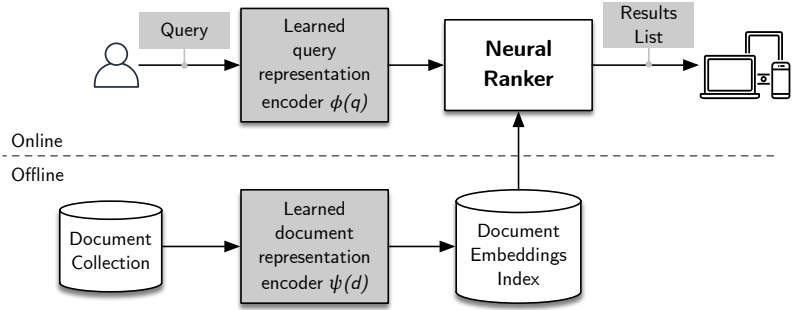


Bi-Encoder (Lecture 5)



	Cross-Encoder	**Bi-Encoder**
Query-doc interaction	Full (all layers)	None (dot product only)
Pre-compute doc embeddings	✗ No	✓ Yes (offline)
Use for retrieval	✗ No (too slow)	✓ Yes (with ANN)
Accuracy	Higher	Lower
Speed at inference	Slow	Fast

Dual Encoder Architecture



Preview: Lecture 5 — Dense Retrieval

The Problem We'll Solve

Cross-encoders can't retrieve. BM25 misses semantic matches.
What if we could do **semantic retrieval** over the full corpus?

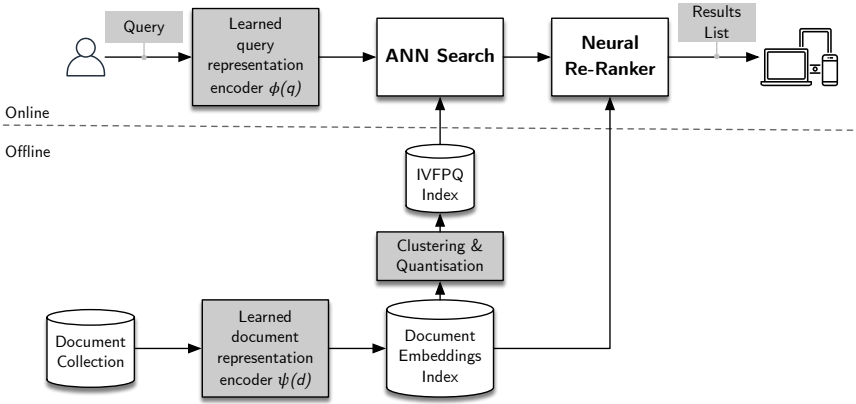
Solution: Bi-Encoders + ANN

1. Encode all documents **offline** with BERT $\rightarrow$ dense vectors
2. Store in ANN index (IVF+PQ from Lecture 2!)
3. At query time: encode query with BERT $\rightarrow$ ANN search $\rightarrow$ top-k

What You'll Learn

- Bi-encoder training: contrastive learning, hard negatives
- DPR, ColBERT (late interaction)
- Hybrid retrieval: BM25 + Dense

Dense Retrieval with ANN



Preview: Lecture 6 — Learning to Rank

Connection to Today

All of today's methods produce **features** for learning to rank:

- BM25(q, d) — sparse relevance score (from Lecture 3)
- DirichletLM(q, d) — language model score (from Lecture 3)
- CrossEncoder(q, d) — BERT relevance score (from today)
- DenseScore(q, d) — bi-encoder similarity (from Lecture 5)
- $|d|$, $|q|$, IDF(q), overlap(q, d), ... — metadata features

Learning to Rank

- Train a model (LambdaMART) to **learn the optimal combination**
- Directly optimizes ranking metrics (NDCG)
- The final piece of the modern retrieval stack

The Complete Modern Retrieval Stack

Stage 1: BM25 retrieves 1000 candidates (L3) *or* Dense retrieval (L5)

↓

Optional: RM3 / LLM expansion improves recall (L3)

↓

Stage 2: Cross-encoder (BERT) re-ranks top 100 (L4 — today)

↓

Stage 3: Learning to Rank combines all signals (L6)

↓

Final ranked list → User or LLM (RAG)

Section 8

Summary and Assignment

Key Takeaways

1. Dense Representations Capture Semantics

Word2Vec/GloVe learn word vectors from co-occurrence. But static embeddings cannot handle polysemy. Contextualized embeddings (BERT) solve this.

2. Transformers Enable Contextualized Understanding

Self-attention lets every token attend to every other token. Multi-head attention captures multiple relationship types. Positional encoding preserves word order.

3. Cross-Encoders Are the Most Accurate Rankers

BERT cross-encoders process query + document jointly. They understand synonymy, negation, and semantic relevance. But they're too slow for full-corpus retrieval.

4. The Two-Stage Pipeline Is the Standard Architecture

BM25 (fast, high recall) $\rightarrow$ Cross-encoder (slow, high precision). RM3 can further improve Stage 1 recall. This is how modern search engines work.

Assignment

Continues from Lecture 3 assignment. Additional tasks:

Part 5: Cross-Encoder Re-ranking (30 pts)

- Add monoT5 re-ranking to your best BM25/RM3 pipeline
- Experiment with re-rank depths (20, 50, 100, 200)
- Report NDCG@10, MAP, and latency per query

Part 6: Analysis (20 pts)

- Identify 3 queries where cross-encoder helps significantly
- Identify 1 query where it fails (recall ceiling)
- For each, inspect top-5 docs and explain *why*
- Discuss: what would dense retrieval (L5) add?

Deliverable

Next Class: Dense Retrieval

Preparation

Read:

- Karpukhin et al. (2020): "Dense Passage Retrieval for Open-Domain QA"
- Khattab & Zaharia (2020): "ColBERT: Efficient and Effective Passage Search"

Skim:

- Reimers & Gurevych (2019): "Sentence-BERT"

What We'll Build

- Bi-encoder retrieval over the full corpus
- Dense + Sparse hybrid retrieval
- Connect to ANN indexing from Lecture 2
- The missing piece: semantic matching at retrieval time

Questions?

Questions?

Office Hours: [Your time]

Email: [Your email]

Key formula to remember:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Backup: BERT Model Sizes

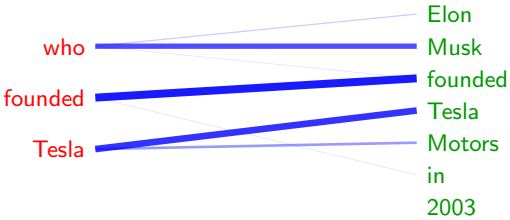
Model	Layers	Hidden	Params	Use Case
BERT-tiny	2	128	4M	Debugging
BERT-mini	4	256	11M	Mobile
BERT-small	4	512	29M	Lightweight
BERT-base	12	768	110M	Standard
BERT-large	24	1024	340M	Best quality

For IR re-ranking:

- BERT-base is the standard (good quality-to-speed ratio)
- BERT-large: $\sim$3$\times$ slower, $\sim$1–2 NDCG points better
- DistilBERT: 40% faster, minimal quality loss (good for production)

Backup: Attention Weight Visualization

Query: "who founded Tesla" **Document:** "Elon Musk founded Tesla Motors in 2003..."



Line thickness $\propto$ attention weight

Observations:

- "who" attends strongly to "Musk" (answer entity)
- "founded" attends to "founded" (exact match) and weakly to "in" (temporal)
- "Tesla" attends to "Tesla" and "Motors" (entity span)

Backup: Beyond BERT — Modern Cross-Encoders

Model	Base	NDCG@10	Key Feature
monoBERT	BERT-large	0.372	Original, simple
monoT5-base	T5-base	0.364	Sequence-to-sequence
monoT5-3B	T5-3B	0.384	Larger $\rightarrow$ better
RankLLaMA	LLaMA-7B	0.392	LLM-based ranking
RankGPT	GPT-4	0.401	Listwise re-ranking

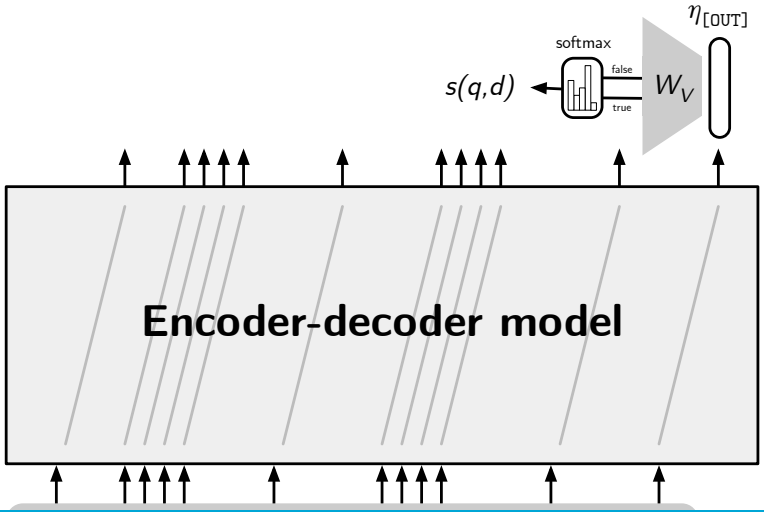
Approximate TREC DL 2020 figures

Trend: Larger language models make better rankers. But at 7B+ parameters, latency becomes prohibitive for most applications.

RankGPT (Sun et al., 2023)

Novel approach: give GPT-4 a *list* of documents and ask it to sort by relevance. No fine-tuning needed. Very expensive but very effective.

T5 for Ranking



Backup: Tokenization Details

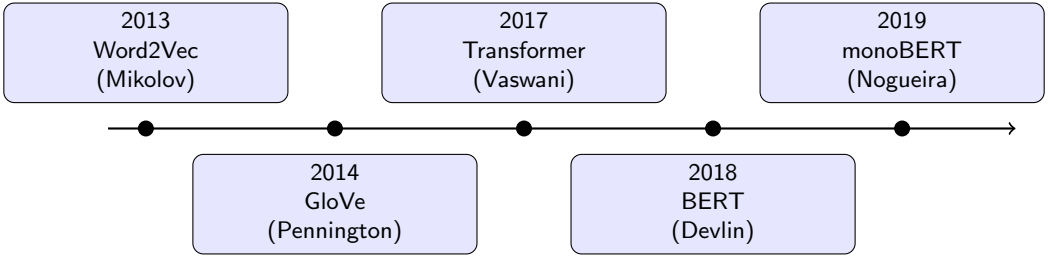
WordPiece Tokenization (BERT):

Input	Tokens
"information retrieval"	["information", "retrieval"]
"transformers"	["transform", "##ers"]
"subword tokenization"	["sub", "##word", "token", "##ization"]
"BM25 algorithm"	["bm", "##25", "algorithm"]

Key properties:

- Vocabulary size: 30,522 tokens
- Common words: single token ("the", "information")
- Rare words: split into subwords ("##" prefix = continuation)
- Max sequence length: 512 tokens (query + document must fit)
- **For IR:** Long documents must be truncated or split into passages

Backup: Historical Context



Key milestones for IR:

- 2013: Dense word representations become practical
- 2017: Attention mechanism replaces RNNs
- 2018: Pre-trained contextual models (transfer learning for NLP)
- 2019: First application to IR re-ranking — large gains over BM25
- 2020+: Dense retrieval, hybrid systems, LLM-based ranking